ELI — What It Really Is, and What It Does for Your Actual Day

Contents

A grounded, human-first portrait of the project — written for the layman and the tech head at once. Every capability claim below is traceable to real code in this repository (see the companion blueprints for the mechanisms). Where something is genuinely impressive, it is said plainly; where the honest limit is the local model, that is said too.

The one-paragraph essence

ELI is a **private artificial intelligence that lives entirely on your own computer.** It talks with you, remembers you across months, runs your machine, sees your screen, reads your files, writes and fixes code, builds documents from your own evidence, and — uniquely — **improves its own source code and can even re-train its own brain on your conversations.** Unlike Siri, Alexa, or ChatGPT, nothing you say to ELI has to leave your house: it is **offline by default, enforced at the network socket itself**, with a switch *you* control. It is not a chatbot bolted onto a cloud API. It is ~140,000 lines of Python that form a complete **cognitive operating system for one person and one machine.** It is **source-available** (PolyForm Internal Use 1.0.0): read, run, and modify it for yourself — redistribution and resale are reserved to the author.

The headline — why ELI is genuinely different

Most "AI assistants" are a thin app talking to someone else's datacentre. ELI inverts every one of those assumptions, and that inversion is the whole point.

1. **It is truly, provably yours.** 100% local. No account, no subscription, no telemetry, no call-home. Your conversations, your memories, your files, your habits — all sit in SQLite databases on *your* drive. The "offline" claim isn't marketing: there is a process-wide network guard (`netguard`) that **fail-closes at the socket layer** — when the Net switch is off, outbound connections physically cannot happen, even if some component tried. You hold the switch.
2. **It owns no brain — so it never goes obsolete.** ELI is **model-agnostic**: no model name or size is hardcoded anywhere on the inference path. It loads whatever local model you give it and auto-detects how to talk to it. The practical magic: **as open local models get smarter, ELI gets smarter for free** — you swap the brain, the body stays. No vendor can deprecate it, price-hike it, or read over its shoulder.

3. **It is honest about itself — because it measures itself.** Ask most AIs "how does your memory work?" and they *invent* a plausible answer. Ask ELI and it reads its own live runtime — actual database row counts, the actual loaded model, the actual pipeline — and reports the truth from **deterministic evidence** rather than inventing it. (Precisely: for self-report/status and router-fast actions the executor's measured result is returned **verbatim in quick mode** and the model only *phrases* it in deeper reasoning modes — it is a mode-gated, partial bypass, not a blanket one; see `grounding_and_evidence.md`.) This deterministic-grounding layer is rare even among serious projects and is the reason ELI can be trusted to describe what it's doing.
4. **It improves itself.** This is the part almost nothing else does locally: ELI logs its own failures, and can **write, syntax-check, import-test, apply, and automatically roll back patches to its own code** — safely, inside its own project. Beyond that, it can **fine-tune its own model on your conversations** (a real LoRA training pipeline using PyTorch/PEFT), so over time it can become measurably more *you-shaped* — all on your hardware, with your data never leaving it.
5. **It has a body, not just a mouth.** ELI doesn't only answer — it *acts*. It opens apps, plays your music, controls windows, takes screenshots, reads your clipboard, sees images and your live screen, reads text out of pictures, and can even **click where your eyes are looking** via webcam gaze tracking. It is an embodied operator on your desktop, not a text box.

If you remember nothing else: **ELI trades a few IQ points (the price of running on your own machine) for total privacy, total ownership, genuine self-honesty, and the ability to grow.** That trade is the product.

For the layman: a day in the life

Picture an assistant you actually *hired* — one who works only inside your home, keeps every note in a locked drawer in your office, and never phones anyone unless you say "okay, you can go online now." That's the feel of living with ELI.

Morning. You sit down and it greets you with a short brief — what happened, what it noticed about your recent work, anything worth your attention. (A background process quietly watches your patterns through the day and assembles this; it never acts on its own without asking.)

While you work. You talk to it by typing or by voice: - *"Open Firefox and my notes."* — done, instantly. - *"Play Vincent's Tale by Ren on Spotify."* — it plays it, on the platform you named, no drifting to the wrong app. - *"What's on my screen?"* — it looks and tells you, and can read text out of any image or screenshot (OCR). - *"Click that"* — with the webcam on, it moves the cursor to wherever your eyes are resting and clicks. Genuinely useful when your hands are full, and a real accessibility win. - *"What's the latest in fusion research?"* — you flip the **Net** switch on, it fetches and **synthesises** the news (it won't just dump raw headlines), then switch it back off and it's fully sealed again.

Real work, not just errands. This is where ELI stops being a gadget: - *"Summarise these three PDFs and this spreadsheet."* — it ingests and analyses them locally. - *"Draft a technical report from these sources."* — its **Report Builder** reads your files as *evidence* and writes a structured document (report, paper, proposal, audit, simulation write-up) with strict discipline: **every claim is tied to your evidence or marked `[source needed]` — it will not fabricate citations or numbers.** - *"Write me a script that does X."* — its coding agent doesn't take one wild guess. It **plans the task, breaks it into a dependency graph, writes it, runs it, tests it, and repairs its own bugs** — and remembers the fix for next time. - *"There's a bug in this file — examine it."* — it runs a tiered scan (syntax, imports, then a careful logic review), shows you what it found ranked by confidence, and only fixes it *after you confirm*, with an automatic undo if the fix breaks anything.

Quietly, over time. It remembers your name, your projects, your interests, your preferences — and those memories are *alive*: an active project stays fresh, an abandoned one fades, so its picture of you reflects *now*, not everything you ever said. It also learns *how* you like to be talked to and adjusts its tone. Offer it a routine ("open my apps every morning") and it'll *propose* automating it — but it never switches a habit on without your yes.

The lived feeling is less “search engine” and more **a calm, private operator sitting beside you who knows your setup, does the legwork, tells you the truth, and gets out of your way.**

For the tech head: the architecture, accurately

~140,000 lines of Python across 351 files. A real cognitive runtime — not an API wrapper:

- **Request pipeline.** A deterministic **router** (166 executor actions, 205 declared capabilities) backed by a **model-grounded intent resolver** that resolves anything the rules miss against that same catalogue (so near-miss phrasings reach real actions instead of a blind chat) → a **14-agent bus** that runs on a *dependency DAG* (topological layers, parallel where independent) with a **calibrated, weight-free confidence aggregator** (each agent’s contribution = evidence quality × payload density × a *learned* per-agent calibration) → a **12-stage retrieval orchestrator** (HyDE query expansion → FAISS vectors + FTS5 keyword + knowledge-graph multi-hop BFS → hybrid merge → a heuristic rerank (lexical × recency × importance; neural cross-encoder is the designed upgrade) → precise context assembly) → one of **five genuinely multi-pass reasoning modes** (chain-of- thought; self-consistency = N samples + a consensus pass; tree-of-thoughts = branch/prune; constitutional = draft→critique; quick) → executor → a stacked output-governance layer.
- **Memory.** Four SQLite stores (user / agent-self-improvement / OS-index / coding-bug-memory) + a FAISS vector index with a local embedder + a knowledge graph + turn-scoped working memory. Facts are **dynamic** — volatile project/interest facts age out unless reaffirmed. Hybrid recall blends vector, full-text, graph, and conversation, then reranks.
- **Inference & hardware.** GGUF via llama.cpp, **fully model-agnostic** (chat template auto-detected by model family — ChatML/Qwen, Llama-3, Mistral). A hardware profiler auto-fits context/GPU-layers/batch to your machine with **adaptive fallback** (observed live: it cascaded through six configs to fit an an 8 GB GPU). A user-tunable **synthesis prompt cap** keeps the small model from degenerating on oversized prompts — exposed in a GUI “Cognition” tab.
- **Self-improvement & learning.** A real failure→analysis→patch loop (generate → syntax-verify → isolated import-test → apply → **auto-revert on failure**, project-scoped, timestamped backups). A full **LoRA fine-tuning pipeline** (torch/PEFT/transformers `Trainer`) that builds datasets from your own conversations with PII redaction and trains adapters locally — human-gated.
- **The grounding/introspection layer (the standout).** A large deterministic subsystem renders self-reports, audits, and identity/memory answers **from live measurements**, not from the model — with an evidence ledger, evidence arbitration, and output validation that rejects/repairs anything the model says that contradicts the gathered facts. Runtime audits now include **live health probes** (plugin manager, memory, agent bus, data-integrity, recent failures).
- **Perception & control.** faster-whisper STT with a wake-word voice gate and output ducking; Piper TTS; local vision-language models (a fast Moondream + a primary VL, hot-swapped with the text model); webcam gaze (MediaPipe + a calibration mapper + One-Euro smoothing at 10 Hz); OCR-based screen-element location; cross-platform OS control.
- **Security.** Offline-by-default enforced at the **socket** (`netguard`, fail-closed) plus a **fail-closed** command/path/app allowlist sandbox (`SecurityManager`) — shell commands are *blocked by default* unless you explicitly allow them. Custom agents are AST-validated and hash-trusted before they can run. A crisis-guard steers the persona on self-harm signals.
- **Extensibility.** A real plugin system (11 built-ins: weather, web, calendar, notes, pomodoro, smart-home, document-reader, web-automation, system-stats, media, TTS) with install/enable/disable, plus user-created custom agents that register live. `capability_sync` keeps the 208-capability manifest *measured* against the actual code, not asserted.

What genuinely works vs the honest ceiling

Fully real, load-bearing today: local model-agnostic inference; offline-by- default socket enforcement; persistent adaptive memory (vector + FTS + graph); the 14-agent calibrated ensemble; the 5 multi-pass reasoning modes; OS control; vision; gaze; STT/TTS; the deterministic self-introspection layer; the coding agent (plan → search → verify → repair → bug-memory); safe self-patching; the evidence-disciplined Report Builder; proactive briefings + habit offers; plugins and custom agents.

Real but you invoke it: LoRA self-fine-tuning (powerful, operator-run, not automatic); web/news/weather (toggle-gated — on when you allow, sealed when you don't).

Built but lightly exercised: an autonomous "world model" / agency layer with a governed proposal console; image *generation* via diffusion (procedural image rendering is the always-available default; diffusion weights are an optional download).

The honest ceiling. The intelligence ELI runs on is a **small local model** — that is the price of total privacy, and it is the one real limit. For the hardest reasoning or broadest world-knowledge, a frontier cloud model will out-think it. A large share of ELI's code exists precisely to *compensate* — to ground the model, catch its mistakes, and stop it confabulating — and that engineering is genuinely excellent. But the model is the bottleneck, and ELI is built so that the moment you drop in a better local model, every part of it gets sharper at once. The hard part — the body around the brain — is already built.

The deeper selling point: ownership

Strip everything else away and this is the thing competitors structurally cannot match. ChatGPT, Claude, Gemini, Alexa, Copilot — every one of them is a tenant arrangement: your data lives on their machines, under their terms, subject to their pricing, their outages, their policy changes, and their ability to read it. **ELI is ownership, not tenancy.** It runs with the internet cable pulled out. It keeps working if the company that inspired it vanishes. It costs nothing to run beyond your own electricity. It answers only to the person at the keyboard.

For anyone who works on sensitive material — research, inventions, personal life, anything you would not paste into a stranger's website — that is not a nice-to-have. It is the entire difference between "an assistant" and "*your* assistant."

Who it's for, and what it means

Right now ELI is a **power-user, single-user, desktop** system — most at home with someone technical on Linux who values privacy and lives on their machine. It is ambitious to the point of audacity in scope, unusually disciplined about telling the truth, and bottlenecked only by the size of brain you can fit on your own hardware.

The most accurate one-line description is not "a chatbot." It is **a private, self-improving, embodied operator for your digital life** — one that remembers you, acts for you, builds with you, tells you the truth about itself, and is wholly, permanently yours. And because the difficult scaffolding is already in place, the trajectory is unusually good: **local models keep improving, and ELI inherits every one of those gains — for free, forever, in private.**

Companion reading: `project_overview.md` (scale + honest engineering verdict), `architecture.md` / `architecture_ascii.md` / `diagrams.md` (the structural map), `grounding_and_evidence.md` (the self-honesty layer), `coding_agent.md` (the frontier coder), `memory.md`, `security.md`, `inference_and_hardware.md`, `learning.md`, `perception.md`.

Update Advisory — 2026-06-07

- Created this session. Numbers verified against the live tree: 126,619 LOC / 336 files / 193 capabilities / 14 bus agents. Corrects an earlier verbal draft that said “15 agents” (the bus has **14**). Expanded ~2× over the original prose to foreground the genuine differentiators (ownership, model-agnosticism, self- honesty, self-improvement, embodiment) while keeping the honest model-ceiling caveat intact.

Update Advisory — 2026-06-07 (continued)

- Counts: **~128.8k LOC / 343 files**; **194 declared capabilities** (155 executor `SUPPORTED_ACTIONS`, 164 routable). **12 main GUI tabs**.
- ELI now **gathers evidence before it generates** (evidence-planner: code/web/ memory/runtime agents) and writes documents through a **multi-stage pipeline** (plan → sections → review→revise) with confidence-driven deeper-tier re-gather — not one shallow pass. Asked about *itself*, it runs the real audits and **summarises** the grounded result (no data dumps, no answering from weights).
- Its **autonomy loop runs** (governed proactive-daemon tick: watches its own code, refreshes its self-model, advances goals into approval-gated proposals).
- Test suite is **green** (2356 passed).